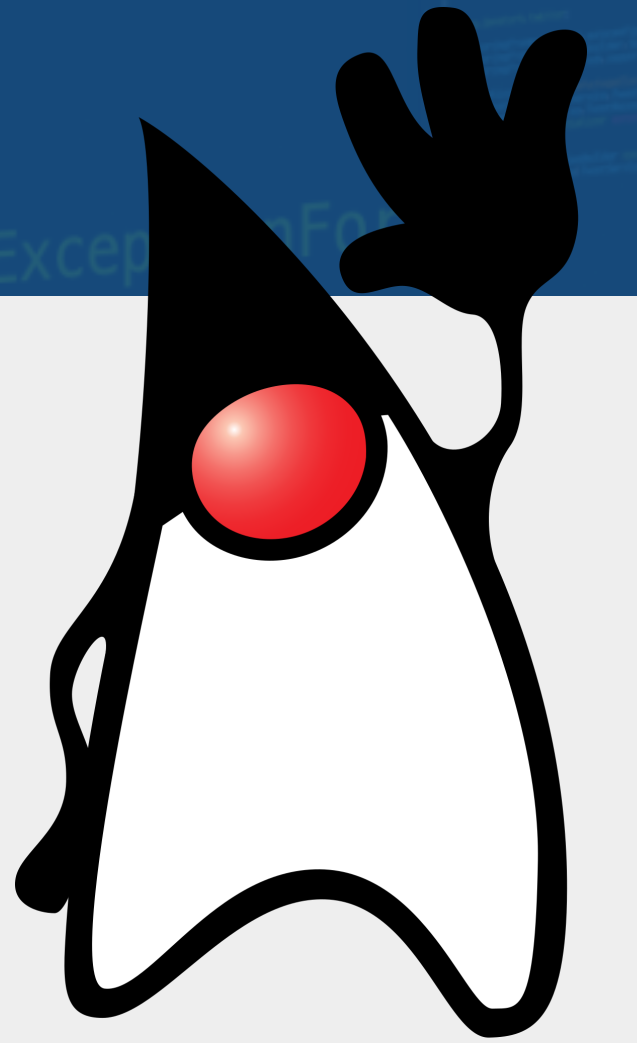


Java Programlama Dili

3. Bölüm

Java'nın Temelleri

Akın Kaldıroğlu





- Söz dizimi, anahtar kelimeler, isimlendirme kuralları, yorumlar vb. konular, her dilin en temel, üzerine diğer konuların bina edildiği detaylarındandır
- Bu bölümde Java'da bu, en temel konular ele alınacaktır

Konular



- **Dilin Simgeleri**
 - Özel Kelimeler
 - Yorumlar
- **İsimlendirme**
 - Kod Geleneği
- **Sabiteler**
- **Ödevler**
- **Bundan Sonra?**



Uygulama

Uygulama



- Al'yi kullanarak çocukken arkadaşım ile oynadığım sayı bulmaca oyununu geliştirdim (number guessing game)
- Al aracıyla geliştirilen kodlar `org.javaturk.jp.game.numberGuessing` paketi altında, farklı Al araçlarının adıyla oluşturulmuş alt paketlerdeki `NumberGuessingGame.java` dosyasındadır
- Bu kodları çalıştıralım ve sonrasında ilerledikçe birlikte inceleyelim

Dilin Simgeleri



Simge - Sembol (Token)



- Bir programda, boşluk (white space) ve yorum (comment) dışında kalan her şey bir **simge** ya da **semboldür** (symbol)
- Sembollere **token** da denir
- Simgeler bir ya da daha fazla karakterden oluşur
- Simge, sembol ya da token, derleyicinin işlediği ve anlamlı olan en küçük program parçasıdır
- Simgeler dildeki farklı yapılara işaret eder

Simge Tipleri - I



- Java'da 5 farklı tipte simge ya da token vardır:
- **İsim** ya da **belirteç (identifier)**: Programda değişken, metot, sınıf vb. yapılara verilen isimlerdir

```
i j counter MyClass Customer calculateCurrentRatio
```

- **Anahtar kelime (keyword)**: Dilde özel anlamı olan kelimelerdir

```
new class int for if volatile synchronized
```

Simge Tipleri - II



- **Değer ifadesi (value expression), sabite (literal):** Sabit olan değerlerdir

```
2 3.1415 true 'c' "Java"
```

- **İşlemci (operator):** Girdilerden bir çıktı üreten özel sembollerdir
- Java'da 38 tane işlemci vardır

```
= + = > < ? : >>> ++ ||
```

Simge Tipleri - III



- **Ayraç (separator):** Kod parçalarını ayıran simgelerdir
- Java'da 12 tane ayraç simgesi vardır

() { } [] ; , @ ::

Dilin Kuralları (Grammar)



- Bir Java programının söz dizimi, yukarıda verilen 5 tür simge, bu simgeler arasındaki boşluklar ve programla ilgili yorumlardan ibarettir
 - Buna **söz** ya da **kelime kuralları (lexical grammar)** denir
- Bir programın geçerli olabilmesi, yani hatasız derlenip çalışabilmesi için, sembollerin belli bir kurala göre kullanılmış olması gereklidir
 - Buna **sözdizimi kuralları (syntactic grammar)** denir

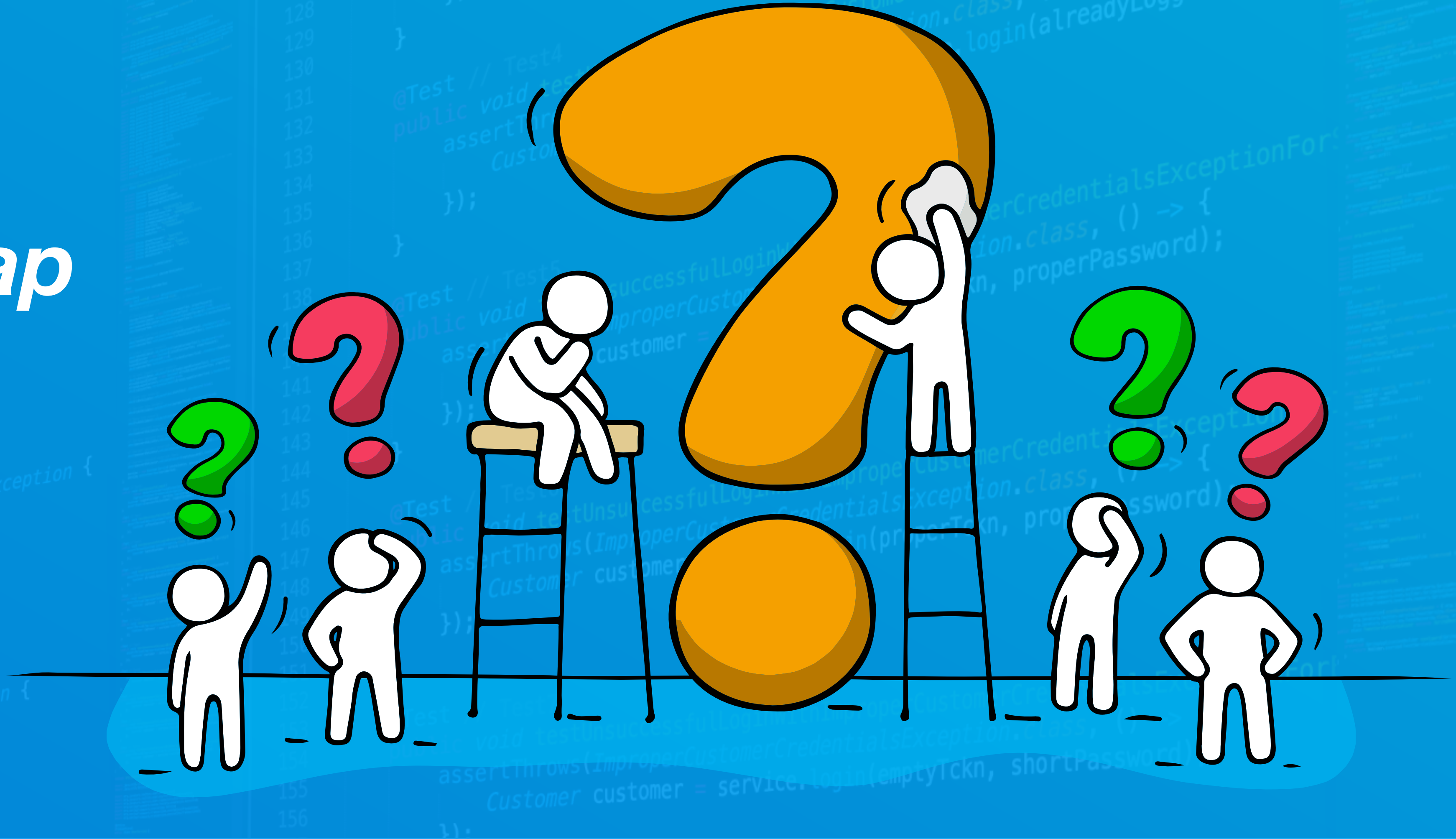


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak farklı türdeki sembollere örnekler bulun

Soru ve Cevap Zamanı!





Özel Kelimeler

Anahtar Kelimeler (Keywords) - I



- Her dilde olduğu gibi Java'da da anlamı ve kullanımı belirlenmiş özel kelimeler vardır; bunlara **anahtar kelime (keyword)** denir
 - Anahtar kelimelere **ayrılmış kelime (reserved word)** de denir
- Anahtar kelimeler ASCII karakterlerinden oluşur
- Java'daki anahtar kelimelere 17. sürümle bir başka tür anahtar kelime daha eklenmiştir:
 - **Bağlamsal anahtar kelimeler (contextual keywords)**

Anahtar Kelimeler (Keywords) - II



- Anahtar kelimeler isim (identifier) olarak kullanılamaz
- Ama bağlamsal anahtar kelimeler ancak belirlenen bağlamlarda anahtar kelime olarak davranır, onun dışında bu özelliği olmayan herhangi bir kelimeye dönüşürler
- Dolayısıyla bağlamsal anahtar kelimeler, belirli bağlamlar dışında isim olarak kullanılabilir

Anahtar Kelimeler (Keywords) - II



- Java'da 51 tane anahtar kelime (keyword) vardır.
- * 1.2, ** 1.4, *** 1.5, **** 9. sürümlerinde eklenmiştir

abstract	continue	for	new	switch
assert **	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum ***	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp *	volatile
const	float	native	super	while
_ (underscore) ****				

Anahtar Kelimeler (Keywords) - IV



- Java'da 17 tane bağlamsal anahtar kelime (contextual keyword) vardır

exports	opens	requires	uses	yield
module	permits	sealed	var	
non-sealed	provides	to	when	
open	record	transitive	with	

- Bağlamsal anahtar kelimelerle, sadece belirli bağlamlarda anahtar kelimedir dolayısıyla isim olarak kullanılamaz
- Bağlamsal anahtar kelimelerle ilgili kısıtlar yeri geldikçe ele alınacaktır

Ayrılmış Kelimeler (Reserved Words)



- **goto** ve **const** en başından bu yana Java'nın **ayrılmış kelimelerindendir (reserved word)** ama dilde hiçbir zaman bir anlama sahip olmamışlar dolayısıyla da kullanılmamışlardır
- En başta olmasına rağmen **strictfp**'nin artık dilde bir anlamı kalmamıştır, kullanılmamaktadır
- **null**, **true** ve **false**, anahtar kelime değildir, Java sabiteleridir (literal) ama onlar da isim olarak kullanılamaz
- Detayları ileride ele alınacaktır

_ Anahtarı Kelimesi



- Java'nın başlangıcından 9. sürümüne kadar _'nin özel bir anlamı yoktu ve isim olarak kullanılabilirdi
- Ama Java 9'da _, ileride olabilecek ihtiyaçlar için ayrıldı ve isim olarak kullanımı yasaklandı, kullanımı derleme hatası olarak değerlendirildi
- Java 22 ile birlikte _, detaylarını sonra göreceğimiz gibi isimsiz değişken (unnamed variable) oluşturmak için kullanılan bir token olarak dilde yerini aldı
- Detay için: <https://openjdk.org/jeps/456>

Selam.java & SelamTest.java



```
public class Selam{  
    String world = "millet";  
    public String selamSoyle(String kime) {  
        String cumle;  
        if(kime != "")  
            cumle = "Selam " + kime + " :)";  
        else  
            cumle = "Selam " + world + " :)";  
        return cumle;  
    }  
}
```

Selam ve SelamTest
sınıflarındaki özel kelimelerin
altı çizilidir

```
public class SelamTest{  
    public static void main(String[] args){  
        Selam nesne = new Selam();  
        String cevap = nesne.selamSoyle("Ali");  
        System.out.println(cevap);  
    }  
}
```

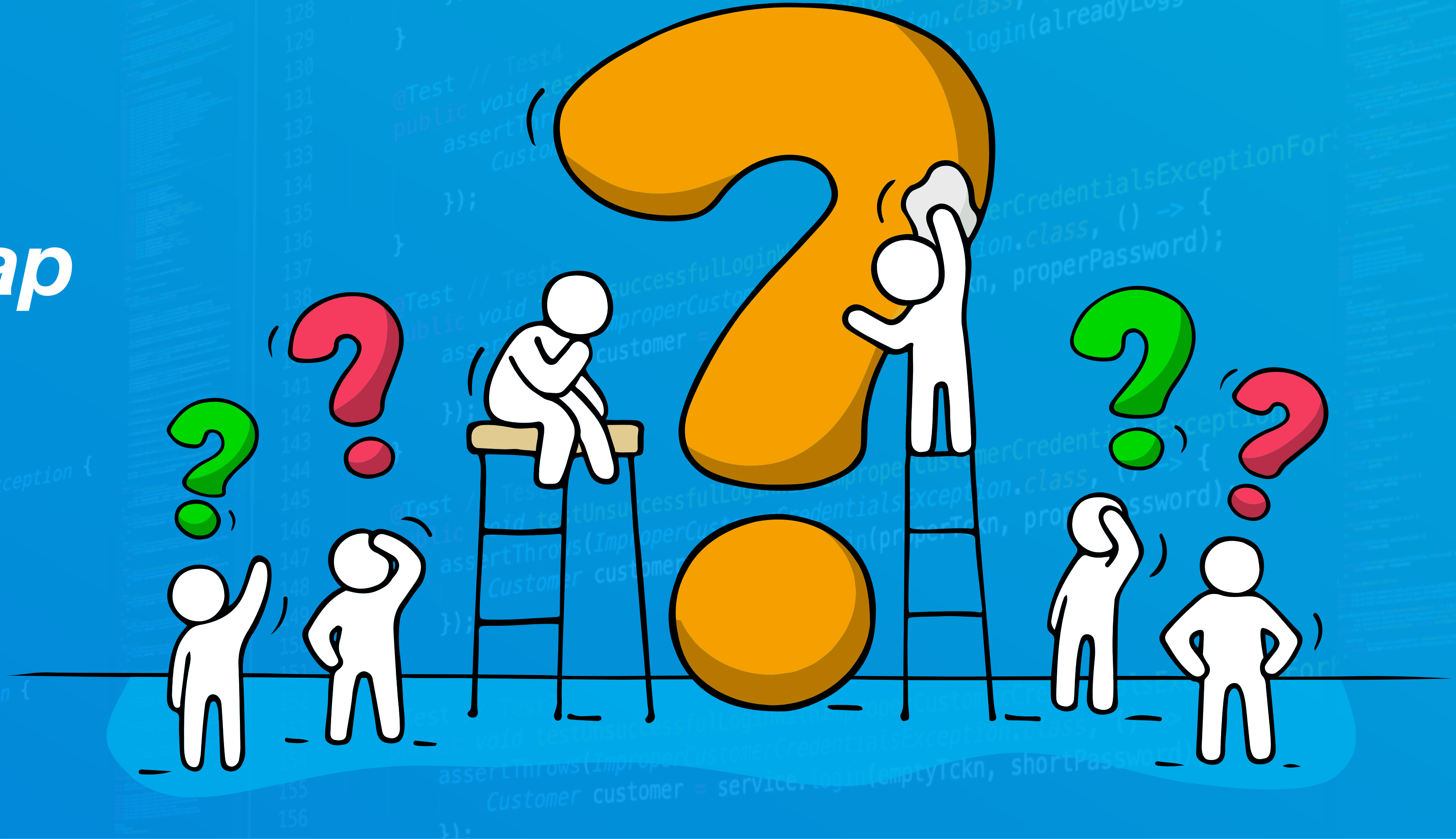


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak özel kelimelere örnekler bulun

Soru ve Cevap Zamanı!





Yorumlar

Yorumlar - I



- Java'da 2 farklı **yorum (comment)** şekli vardır:
- **//** (çift yatık çizgi) (**end-of-line comment**) bulunduğu yerden itibaren satırı yorum yapar:

```
double area = 3.14 * r * r; // 3.14 is PI
// Bu satır tamamen bir yorum satırıdır
```

- **/* */** (**block/traditional comment**) bir ya da daha fazla satırı, blok olarak yorum yapar:

```
/*
 * Bu, birden fazla satırı yorum yapmanın yoludur.
 * Genelde "yatık çizgi yıldız" ile "yıldız yatık
 * çizgi" farklı satırlarda olur.
 */
```

Yorumlar - II



- Yorumlar sınıf içinde ya da dışında kullanılabilir
- Bir ifade içinde `//` olamaz, `/* */` olabilir ama okunurluğu çok bozar
- Karakter ya da `String` sabiteleri (literal) içinde kullanılırsa yorum olarak görülmezler
- Yorumlar birbirlerini kesmemelidir
 - `//` içinde `/* */` ya da `/* */` içinde `//`'nin hiçbir özel anlamı yoktur

JavaDoc Yorumları



- **/**** ***/** **Javadoc** ile dokümantasyon oluşturmak amacıyla kullanılan yorumdur
- Javadoc hakkında ileride daha geniş bilgi verilecektir

```
/**
 * Bu birden fazla satırı <b>Javadoc</b> yorumu yapmanın
 * yoludur. Javadoc, bir Java aracı ve iç
 * dokümantasyon oluşturma formatıdır.
 * @see http://www.javaturk.org
 * @see https://www.oracle.com/java/technologies/javase/javadoc-tool.html
 * @author Akin Kaldiroglu
 */
```

Yorumlar - III



- `//` ve `/* */` kod içi dokümantasyon, Javadoc ise API dokümantasyonu için kullanılır

Comments & WeirdComments



- `org.javaturk.ipj.ch03.Comments`
- Bu örnekte farklı yorum tipleri verilmektedir
- JavaDoc yorumlarındaki `@author` ya da `@param` gibi etiketlere dikkat edin
- Ayrıca `WeirdComments` sınıfındaki yorum kullanımlarını inceleyin

Dokümantasyon Yapmak-Yapmamak



- JavaDoc ile yazılan API dokümantasyonu dışında, kod içi dokümantasyon yerine daha anlaşılır kod yazmaya gayret edin
- Anlaşılır kodun dokümantasyona ihtiyacı azdır!
- Kodu anlaşılır yapmanın en temel yöntemi, dokümantasyon değildir, anlaşılır, temiz kod (clean code) yazmaktır

```
// Stupid code, annoying comment!  
double tp = p * 1.18; // Multiply price with 1.18  
  
// Stupid code, stupid code-fixing comment!  
double tp = p * 1.18; // 0.18 is the ratio for VAT  
  
// Good code, no comment!  
double totalPrice = price * (1 + VALUE_ADDED_TAX_RATIO);
```

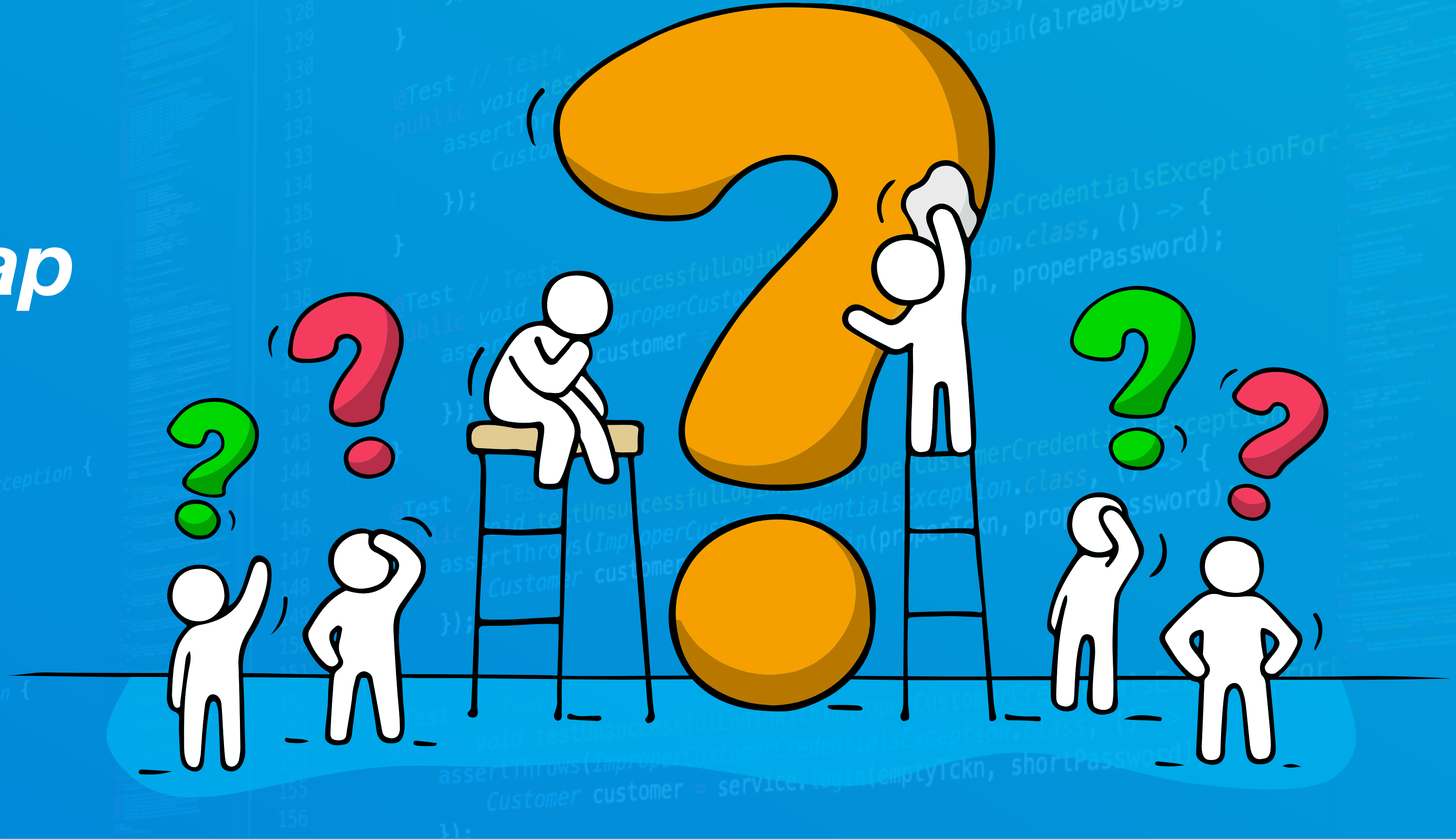


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak yorumlara örnekler bulun

Soru ve Cevap Zamanı!



isimlendirme



İsimler / Belirteçler (Identifiers)



- Bir programdaki isimlere (name), **belirteç (identifier)** denir
- Java programlarındaki belirteçler, modül, paket, sınıf, enum, record, arayüz, metot, blok ve değişken isimlendirmek için kullanılır
- Her dilde olduğu gibi Java'da da isimlendirmeyeyle ilgili kurallar vardır

Küçük - Büyük Harf Ayırımı



- Java'da küçük-büyük harf ayrımı vardır:
 - **Araba** ve **araba** farklı isimlerdir
- Bu durum, böyle bir ayrıma sahip olmayan programlama kültüründen gelenlere ilk başta zorluk çıkarabilir
- Dilde nerede küçük harf nerede büyük harf kullanılması gerektiği **Kod Geleneği (Code Conventions)** tarafından belirlenir
- Java kültüründe detaylı bir kod geleneği vardır

İsimlendirme Kuralları - I



- Java'da isimlendirme kuralları şunlardır:
 - Anahtar ve ayrılmış kelimeler isim olarak kullanılamaz
 - **null**, **true** ve **false** da isim olarak kullanılamaz
 - Bağlamsal kelimeler, ancak kendilerine özel bağlamlar dışında isim olarak kullanılabilir
 - Bu kelimeler yeri geldikçe ele alınacaktır

İsimlendirme Kuralları - II



- Ayrıca:
- İsimler sadece harf ile başlayabilir
- İsimler bir rakam ile başlayamaz ama sonraki karakterlerde rakam olabilir
- İsimler, " ' . , ; : ! ? / \ - | # & gibi **noktalama işaretleri (punctuation marks)** içeremez ama **_** yani alt çizgi (underscore) içerebilir
- İsimler boşluk (space) içeremez

İsimlendirme Kuralları - III



- *İsimler sadece harf ile başlayabilir* kuralının iki istisnası vardır:
 - İsimler, \$, €, ¢, ¥, £, ₺ gibi bir para birimi sembolleri (currency symbols) ile başlayabilir ama bu yaygın bir kullanım değildir
 - \$, Java tarafından otomatik olarak üretilen isimlerin başında kullanıldığından, kaçınılmasında fayda vardır
- İsimler, _ yani alt çizgi (underscore) ile de başlayabilir
 - Java kod geleneğinde _'nin çok kısıtlı bir kullanımı vardır

Alt Çizgi (Underscore) _ - I



- Java 9'dan itibaren sadece alt çizgi de artık bir anahtar kelimedir ve isim olarak kullanılamaz olmuştur
- Zaten Java 8, sadece bir isim olarak kullanıldığında, *use of '_' as an identifier might not be supported in releases after Java SE 8* şeklinde uyarı veriyordu
- Daha önce bahsedilmişti, Java 22 ile birlikte , detaylarını ileride göreceğimiz gibi isimsiz değişken (unnamed variable) oluşturmak için kullanılan bir token oldu

```
int _ = 5;
```

Alt Çizgi (Underscore) _ - II



- Öte taraftan _ Java'da isimlerin içinde bile sık kullanılan bir karakter değildir
- _, isimlerin içinde kullanılabilir ama Java'nın kod geleneğinde sadece sabitelerde bulunur, `VALUE_ADDED_TAX` gibi,
- `_name`, `first_name` gibi kullanımlar Java'da yoktur, siz de kullanmayın
- Ayrıca ileride ele alacağız _, kolay okunması amacıyla, `1_000_000` gibi, sayılarda rakamların arasında kullanılabilir

İsimlendirme (Naming)



- Daima anlamlı isimler kullanın:
- Okunduğunda ne işe yaradığını, onunla ne yapmak istediğinizi yansıtan isim kullanmak önemlidir
 - intention revealing names
- Java'da isimlerin uzunluklarında bir sınır yoktur
- Kodlanmış, kısa isimler yerine anlamlı gerekirse uzun isimler daha iyidir

<code>nofinstl</code>	<code>⇒</code>	<code>numberOfInstallments</code>
<code>prc</code>	<code>⇒</code>	<code>price</code>
<code>iaca</code>	<code>⇒</code>	<code>interestAccruingCheckingAccount</code>

Unicode



- Java karakterleri **Unicode** olduğundan Türkçe, Arapça, İbranice, Çince, Hintçe, Japonca, Almanca vb. dillerdeki harfler de kullanılabilir
- Fakat yine de isimlerin İngilizce olarak yazılması pek çok açıdan daha uygun olacaktır

İsimlendirme Örnekleri



- Aşağıdaki isimler geçerlidir:

```
sayı Sayı mädchen weiß değer אדם رجل 男 $price
```

- Aşağıdaki isimler geçerlidir ama gelenekte yeri yoktur yani bir Java developer böyle isimler vermez:

```
sum_$ $$_100 super_duper_price _l_prc
```

- Aşağıdaki isimler geçerli değildir, Java derleyicisi hata verir:

```
48liler all/clear kaybol-buradan true true?
```

ValidAndInvalidNames.java



- `ValidAndInvalidNames`

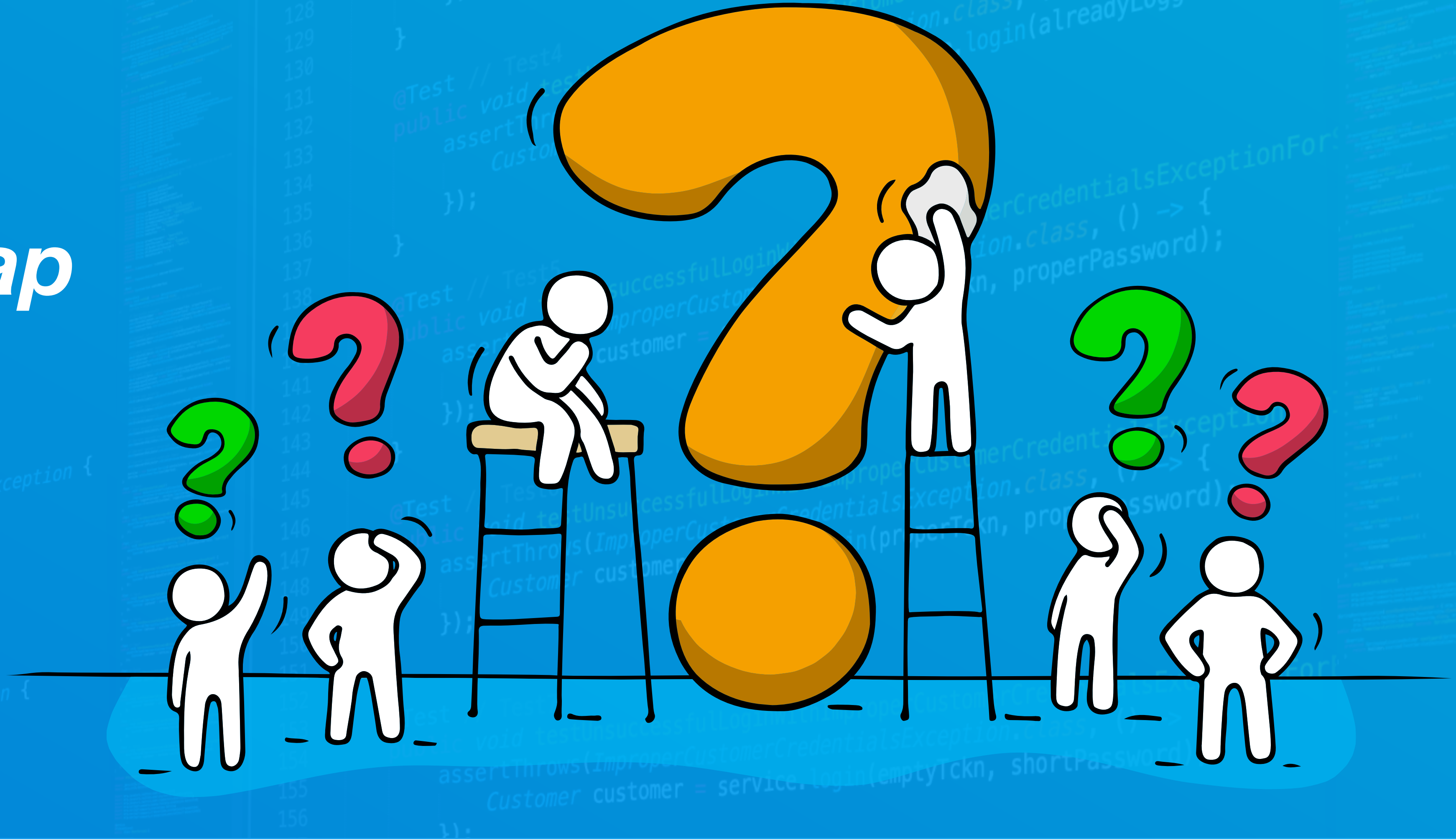


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak isimlere örnekler bulun

Soru ve Cevap Zamanı!





Kod Geleneği



- **Kod Geleneği (Code Conventions)**, bir dili kullananlar tarafından üzerinde hemfikir olunmuş ve kodun şekilsel olarak nasıl yazılacağını, büyük-küçük harf, boşluk, satır vb. görüntüsel veya estetik durumlarını belirleyen kurallar bütünüdür
- Kod geleneğine uyulmazsa derleyici hata vermez çünkü dilin kurallarından değildir ama çevrenizdeki geliştiriciler tarafından kınanırsınız
- Dolayısıyla kod geleneğine uymamak *ayıp* addedilir

Camel Case - I



- İsimlendirme, kod geleneğinin önemli bir parçasıdır
- Java'da isimlerde **Camel Case** (CC) yaklaşımı kullanılır yani birden fazla kelime varsa kelimeler birleştirilir, aralarına _ konmaz
- CC'in iki türü vardır:
 - **Upper CC**'de ilk kelimenin baş harfi büyüktür
 - **Lower CC**'de ilk kelimenin baş harfi küçüktür

```
LimitedLiabilityCompany
```

```
InterestAccruingCheckingAccount
```

```
limitedLiabilityCompany
```

```
interestAccruingCheckingAccount
```

Camel Case - II



- Sınıf, arayüz, enum, record vb. tip isimleri, Upper CC ile yazılır:

```
CheckingAccount  
RentalAccountInformation
```

- Değişken isimleri Lower CC ile yazılır:

```
firstName  
accountNumber  
returnRatioForRentals
```

- Metot isimleri Lower CC ile yazılır:

```
getAccountNumber()  
printReturnRatioForRentals()
```

Selam.java & SelamTest.java



```
public class Selam{
    String world = "millet";
    public String selamSoyle(String kime) {
        String cumle;
        if(kime != "")
            cumle = "Selam " + kime + " :)";
        else
            cumle = "Selam " + world + " :)";
        return cumle;
    }
}
```

Selam ve SelamTest
sınıflarındaki isimlere dikkat
edin, hepsinin altı çizilidir

```
public class SelamTest{
    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSoyle("Ali");
        System.out.println(cevap);
    }
}
```

Naming



- **Naming**
- Bu örnekte bir Java sınıfında, sınıf, değişken ve metot gibi farklı isimlendirmelerin, kod geleneğine uygun olarak nasıl yapıldığı gösterilmektedir

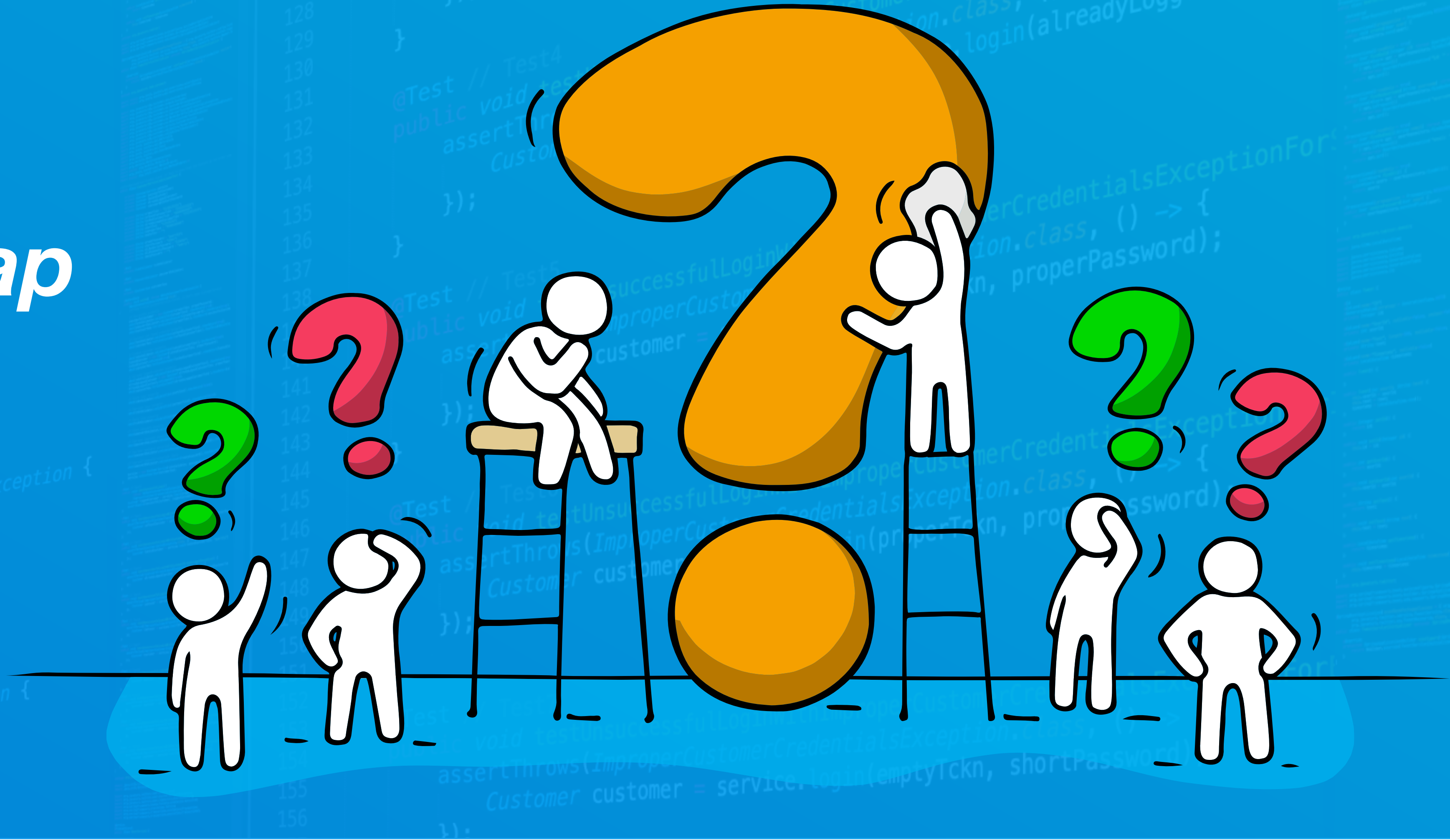


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak isimlerle ilgili kod geleneği kullanımına örnekler bulun

Soru ve Cevap Zamanı!



Sabitleler



Sabiteler (Literals) - I



- **Sabiteler (literals)**, programlarda kullanılabilecek olan ve değişmeyen değerleri ifade eden kelimeler veya sembollerdir
- Java'da dört farklı tipte sabite vardır:
 - Sayı (number)
 - Mantıksal sabite (logical literal)
 - Karakter (character)
 - Sıfır ya da daha fazla karakterden oluşan karakter dizisi, metin (string)

Sabiteler (Literals) - II



- **Sayılar:** Tam sayılar (integer) ve tam olmayan sayılar (rasyonel sayılar ile irrasyonel sayılar) sayılar (floating-point):

2000 -7 0.5 3.1415 1.6E-19

- **Mantıksal (logical) sabiteler:** Doğruluk değerini gösterirler, doğru ve yanlış karşılık gelecek şekilde sadece iki tanedir

true false

Sabiteler (Literals) - III



- **Karakterler:** İki tane tek kesme arasında gösterilir :

'a' '*' 'ş' '-' ' ' '!'

Sabiteler (Literals) - IV



- **Metin, karakter dizisi (string):** İki tane çift kesme arasında bulunan sıfır ya da daha fazla karakterdir:

```
"Java"    "*** Names ***"    "3.1415"    "x" " " " " ""
```

- Ayrıca, çok satırlı metinleri ifade etmek için kullanılan, iki tane üçlü tırnak, `""" """`, arasında sıfır ya da daha fazla karakter içeren metin blokları (text blocks) da vardır:

```
String poem = """
                Leylim Ley
Döndüm daldan düşen kuru yaprağa
Seher yeli dağıt beni kır beni
Götür tozlarımı burdan uzağa
Yarin çıplak ayağına sür beni
                Sabahattin Ali""";
```

Sabiteler - V



- Bir de sadece referanslara atanabilen `null` sabitesi vardır

Sabiteler ve Tipler



- İleride bu sabitelerin farklı türde veriler olduklarını anlayacağız
- Dolayısıyla bu farklı türde veriler için farklı tipler (types) söz konusu olacak

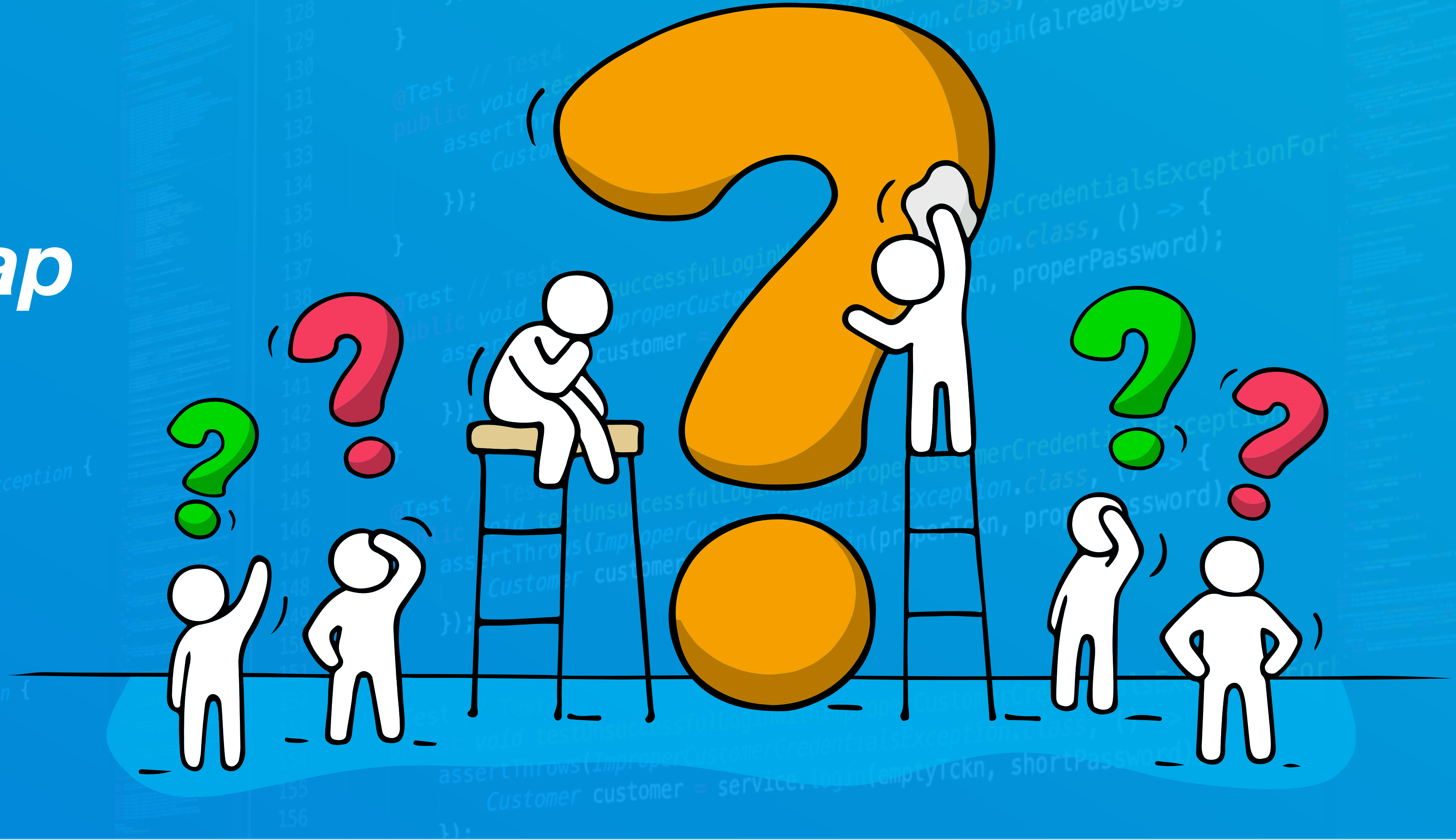


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak sabitelere örnekler bulun

Soru ve Cevap Zamanı!



Ödevler



Ödev - I



- `hw.ch03` paketindeki `Calculator.java` dosyasındaki kodun sembollerini, isim, anahtar kelime, değer ifadesi ya da sabite, işlemci ve ayraç olarak belirleyin

Ödev - II



- Aşağıdaki isimlerden geçerli olanları belirleyin, geçersiz olanların ihlal ettiği kuralı belirtin:
 - firstName
 - 2ndPlace
 - \$amount
 - user-name
 - _total
 - when
 - MAX_VALUE
 - email@address
 - αβγ
 - while

Ödev - III



- `hw.ch03` paketindeki `NamingProblems.java` dosyasındaki isimlerden Camel Case'e uymayanları veya iyi olmayanları belirleyin ve düzeltin

Bundan Sonra?



Bundan Sonra?



- Bir sonraki bölümde Java'nın kaynak kod yapısını (source code structure) öğreneceğiz
- Sonraki bölümlerde ise tipler ve değişkenleri ele alacağız
- Önce ilkel tipler, sonra karmaşık tipler ve sonra da bu tiplerden oluşturulan değişkenleri işleyerek ilerleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

